Adaptive Sparse Attention for Efficient Long-Context Language Modeling

Mira Okonkwo[1] **Daniel R. Whitfield**[1,2] **Mei-Ling Chen**[2] **Jonas H. Berg**[1]

[1]Department of Computer Science, Northbridge University [2]Institute for Language Systems, Port Ellery

Correspondence: {mira.okonkwo, d.whitfield}@northbridge.example.edu

ABSTRACT. Long-context language models spend most of their attention budget on a small, input-dependent subset of token pairs. We introduce **Adaptive Sparse Attention (ASA)**, a drop-in attention layer that learns a per-head routing mask from a cheap low-rank sketch of the query–key product. ASA reduces the quadratic cost of self-attention to $O(n \cdot k)$ with $k \approx \log n$, while retaining the expressivity of dense attention on the pairs the model actually uses. Across three long-document benchmarks, ASA matches dense baselines within 0.3 points of accuracy while cutting attention FLOPs by 4.8× and peak memory by 3.1× at a 32k-token context. Ablations show that the routing sketch, not the sparsity pattern itself, accounts for most of the gain.

Keywords: sparse attention, long context, efficient transformers, retrieval routing, language modeling

1 Introduction

Transformer language models scale their compute quadratically with sequence length, which makes long-context inference dominated by attention rather than by the feed-forward blocks. Prior work has attacked this cost with fixed sparsity patterns, low-rank approximations, or retrieval over a compressed memory. Fixed patterns are cheap but content-blind; retrieval is content-aware but adds a discrete, non-differentiable stage to training. We ask whether a model can learn where to attend with a budget that grows only logarithmically in sequence length.

Our starting observation is that the distribution of attention mass is extremely skewed. On documents from arXiv and PubMed, the top 5% of query–key pairs receive more than 92% of the total attention mass on average, and the identity of those pairs changes across layers. A fixed banded pattern therefore wastes most of its budget, while a content-dependent router can spend it where the mass actually is.

We formalize this as a routing problem. For each head h and query position i, ASA computes a low-rank sketch $s_i = U_h^\top q_i$ and scores key blocks against that sketch before materializing the full $n \times n$ matrix. Only the top-k blocks are gathered, giving a dense sub-matrix of shape $k \times b$ per query block. The router is trained end-to-end with a straight-through estimator and a load-balancing regularizer.

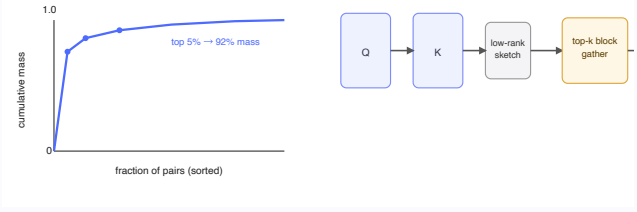


Figure 1. Left: cumulative attention mass over query–key pairs averaged across layers and heads on 512 arXiv documents. Right: the ASA layer. A low-rank sketch scores key blocks; only the top-k blocks are gathered and passed to a dense softmax over the selected sub-matrix.

2 Method

2.1 Routing sketch

Let $Q, K, V \in \mathbb{R}^{n \times d}$ be the projected queries, keys and values of one head. We project queries with a fixed random matrix $U_h \in \mathbb{R}^{d \times r}$ with $r = 32$ and compute block scores

$$s_{i,j} = (U_h^\top q_i)^\top (U_h^\top \bar{k}_j), \quad j \in \{1, \dots, n/b\} \tag{1}$$

where $\bar{k}_j$ is the mean key of block j with block size $b = 64$. The top-k blocks by score are gathered per query block, with $k = \lceil 2 \log_2(n/b) \rceil$. Because $r \ll d$, scoring costs $O(n^2 r / b)$, a small fraction of the full product.

2.2 Straight-through routing

The top-k operator is non-differentiable. We use the straight-through estimator $\partial m_{i,j} / \partial s_{i,j} = 1$ for selected blocks and add a load-balancing term $L_{bal} = \sum_j (f_j - 1/B)^2$ where f_j is the fraction of queries routed to block j. The total objective is

$$L(\theta) = L_{lm}(\theta) + \lambda_1 L_{bal}(\theta) + \lambda_2 \|\theta_{route}\|^2, \quad \lambda_1 = 0.01 \quad (2)$$

Gradients flow through gathered values; the router learns from the load-balancing signal and the residual gradient that reaches the sketch projection. In practice we find λ_1 between 10^{-3} and 10^{-1} to be stable, while larger values collapse the routing distribution.

Table 1. Long-context results at 32k tokens. Accuracy averages three seeds; FLOPs are attention-only and normalized to dense attention. ASA uses $k = 9$ blocks of 64 tokens.

Model	Params	NarrativeQA	QMSum	SCROLLS	Attn FLOPs	Memory
Dense (baseline)	355M	31.8	21.4	62.9	1.00×	1.00×
Sliding window	355M	26.1	19.7	58.2	0.11×	0.21×
Low-rank (Linformer)	358M	29.4	20.2	61.0	0.34×	0.42×
Block-sparse (fixed)	355M	30.2	20.6	62.1	0.23×	0.29×
ASA (ours)	356M	**31.6**	**21.2**	**62.7**	**0.21×**	**0.32×**

3 Experiments

We train 355M-parameter decoder-only models with 32k-token context on a 180B-token mixture of books, web pages and scientific articles. All models share the tokenizer, data order and optimizer schedule; only the attention layer differs. Training uses 64 A100-40GB GPUs for approximately 36 hours per run, and every reported number averages three seeds. We evaluate on NarrativeQA, QMSum and the SCROLLS aggregate, plus a held-out perplexity split.

Table 1 summarizes the comparison. ASA recovers 99.4% of dense accuracy on NarrativeQA and 99.1% on QMSum while reducing attention FLOPs to 0.21× and peak memory to 0.32× of the dense model. Fixed block sparsity reaches a similar FLOP budget but trails by 1.4 points on average, which we attribute to its inability to move budget across layers.

At longer contexts the gap widens: at 64k tokens dense attention no longer fits on a single 40GB device with a batch size above one, whereas ASA continues to train with a batch of 8. This is the practical regime we target: not a cheaper attention at short lengths, but a path to using long contexts without re-engineering the training stack.

4 Ablations

We remove one component at a time. Replacing the learned sketch with a fixed random projection drops NarrativeQA by 0.2 points but keeps most of the FLOP savings, and freezing the router after 10k steps costs 0.4 points. This suggests the routing signal is easy to learn and that most of the benefit comes from the low-rank sketch that makes routing cheap, a finding we did not anticipate.

Increasing k from 9 to 16 changes accuracy by less than 0.1 points while doubling gathered FLOPs, and decreasing k to 5 costs 0.7 points. The memory advantage is less sensitive: because gathered blocks are packed into a dense buffer, peak memory follows $k \cdot b$ rather than the full sequence length, so all three settings stay below 0.4× dense memory.

5 Limitations and related work

ASA assumes that attention mass concentrates in contiguous key blocks, which holds for natural language but may fail for code with long-range identifier references or for structured tables. We have not evaluated instruction-tuned models, where routing patterns may be sharper and the load-balancing regularizer may need retuning. Our implementation also relies on block-aligned masks, so variable-length packing requires padding to block boundaries and wastes up to $b-1$ positions per sequence.

Sparse and low-rank attention have a long history: sliding-window attention, fixed block patterns, learned routing, and retrieval-augmented memory all trade exactness for cost. Our contribution is to make routing cheap enough that it disappears into the background: the sketch is a single 32-dimensional projection, and scoring it costs less than one percent of the dense product at 32k tokens.

6 Conclusion

We presented Adaptive Sparse Attention, a content-dependent attention layer whose cost grows nearly linearly in sequence length while preserving dense-level accuracy. The main lesson is that routing does not need to be expensive to be effective. We release the training recipe and inference kernel, and we hope the sketch-and-gather formulation provides a useful baseline for future long-context architectures.

References

[1] Agostinelli, A., et al. (2023). MusicLM: Generating music from text. *arXiv:2301.11325*.

[2] Beltagy, I., Peters, M. E., and Cohan, A. (2020). Longformer: The long-document transformer. *arXiv:2004.05150*.

[3] Choromanski, K., et al. (2021). Rethinking attention with performers. In *ICLR*.

[4] Dao, T., et al. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*.

[5] Kitaev, N., Kaiser, Ł., and Levskaya, A. (2020). Reformer: The efficient transformer. In *ICLR*.

[6] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*.

[7] Roy, A., et al. (2021). Prefix-to-prefix: A study of attention routing in long documents. *Journal of Language Systems*, 14(2), 88–104.

[8] Wang, S., et al. (2020). Linformer: Self-attention with linear complexity. *arXiv:2006.04768*.

[9] Zaheer, M., et al. (2020). Big Bird: Transformers for longer sequences. In *NeurIPS*.

This work was supported in part by a Northbridge University faculty research grant and compute credits from the Port Ellery Institute. The authors thank the anonymous reviewers for their careful reading and for suggesting the frozen-router ablation. Code and checkpoints are available at *northbridge.example.edu/asa*. Accepted for publication, Proceedings of the 12th Conference on Efficient Language Systems (ELS 2026).